14.1.2 Graphs as Algebraic Types

There is another aspect, in graph representation, that is orthogonal to the way we store edges: consistency.

To be fair, inconsistencies are much easier to happen with the adjacency list representation, but still possible, in certain situations, even using adjacency matrix.

The problem is the following: regardless of its representation, consider the following graph: `G=([1,2], [(1,2), (1,3), (2,2)])`.

The graph G has two vertices, `[1,2]`, but it has an edge whose destination is the vertex "3"; this can happen for any reason, for instance sloppiness in deleting vertex "3", or an error while adding edges.

Moreover, the graph has a loop, the edge `(2,2)`: what if `G` was supposed to be a simple graph, without loops?

Of course, we can add validation to our `Graph` class' methods to prevent these situations, but the data structure itself can't guarantee that these errors won't happen.

To overcome these limitations, we can define our graphs as an algebraic type[1]; this way, we define graphs as either:

- The empty graphs;
- A singleton, a single vertex with no edges;
- The connection between two graphs `G` and `G'`: we define one or more edges whose source is in `G` and destination in `G'`.